

ЗАДАЧА ЗА ФИГУРИ – override vs. new допълнена

Демонстрираме, че new не е еквивалентно на override.

Когато имаме метод в родителя, който **не е virtual**, но искаме да го променим в наследника, ние не може да използваме override, но в такива случаи използваме скриване с **new**.

```
public class Parent
{
    public void Method() => Console.WriteLine("Parent");
}

public class Child : Parent
{
    public new void Method() => Console.WriteLine("Child");
}
```

В примера, знакът **=>** просто е съкратен запис, който спестява изписването на къдрави скоби {}, по същия начин, както използвахме този съкратен запис във валидациите на get.

Може да имаме последователност от наследявания!

Използваме new когато:

- * Базовият метод не е virtual и не може да се override, т.е. не искате полиморфизъм за този метод
- * Искате съвсем различна имплементация
- * Работим със затворени (sealed) класове или методи

base. извика родителски метод вътре в друг метод, за да го разшири с още функционалност, докато new директно го подменя.

- base. е за **разширяване** - "извикай базовата логика и добави нещо"
- new е за **замяна** - "не искам да използвам базовата логика, имам нова"

```

public class ThirdPartyLibraryClass
{
    public void Process() // Не е virtual, не може да се override
    {
        // Стара логика
    }
}

public class MyExtendedClass : ThirdPartyLibraryClass
{
    // Единствената възможност - скриваме метода
    public new void Process()
    {
        // Нашата нова логика
        PreProcess();
        base.Process(); // Може да го извикаме, ако искаме
        PostProcess();
    }
}

```

Сега нека тестваме различни възможности как биха реагирали пренаписванията, включително ако хвърлим override след sealed, ако използваме new след sealed, ако използваме new за виртуален метод вместо override и коментираме резултатите:

```

public class Shape
{
    4 references
    public virtual void Draw() { Console.WriteLine("Drawing a generic shape..."); }
}

2 references
public class Circle : Shape
{
    3 references забранява за наследниците на Circle, но за наследниците на родителя Shape може
    public sealed override void Draw() { Console.WriteLine("Drawing a circle..."); }
}

1 reference
public class Square : Circle наследник на наследник
{
    //public override void Draw() { Console.WriteLine("Drawing a square..."); } // забранено
    0 references
    public new void Draw() { Console.WriteLine("Drawing a square..."); } // отпечатва родителя circle
    не
}

1 reference
public class Rectangle : Shape родител
{
    0 references
    public new void Draw() { Console.WriteLine("Drawing a rectangle..."); } // отпечатва родителя
    не
}

1 reference
public class Triangle : Shape
{
    3 references разрешено родителя
    public override void Draw() { base.Draw(); Console.WriteLine("Drawing a triangle..."); } // отпечатва родителя + triangle
    ДА
}

```

Изводи:

- 1) Когато родителят има метод с **virtual**, наследниците могат да го презапишат (променят) с **override**.
- 2) използваме **new**, когато методът на родителя не е virtual, съответно не можем да го презапишем с **override**, но можем да го скрием (засенчим).
- 3) Използваме **sealed** пред **override**, когато искаме наследниците на наследника да не могат да променят повече. Другите наследници на родителя, обаче, могат да променят.
- 4) За разширение, т.е. допълване с още функционалности, на даден **override** метод, вътре в него извикваме родителския (базов) с **base.**
- 5) Неправилно и безсмислено е да използваме **new** вместо **override** за виртуален метод.

```
public class Shape
{
    public virtual void Draw() { Console.WriteLine("Drawing a generic shape..."); }
}

public class Circle : Shape
{
    public sealed override void Draw() { Console.WriteLine("Drawing a circle..."); }
}

public class Square : Circle
{
    //public override void Draw() { Console.WriteLine("Drawing a square..."); }
    // забранено
    public new void Draw() { Console.WriteLine("Drawing a square..."); } //
    отпечатва родителя circle
}

public class Rectangle : Shape
{
    public new void Draw() { Console.WriteLine("Drawing a rectangle..."); } //
    отпечатва родителя
}

public class Triangle : Shape
{
    public override void Draw() { base.Draw(); Console.WriteLine("Drawing a triangle..."); } // отпечатва родителя + triangle
}
```

В **Main()** създаваме **полиморфичен масив** с няколко нови обекта - затова типът във **foreach** е **var**:

```
static void Main(string[] args)
{
    Shape[] shapes = { new Circle(), new Square(), new Rectangle(), new Triangle() };
    foreach (var s in shapes) s.Draw();
}
```

```
Shape[] shapes = { new Circle(), new Square(), new Rectangle(), new Triangle() };
    foreach (var s in shapes) s.Draw();
```

Microsoft Visual Studio Debug Console

```
Drawing a circle...  
Drawing a circle... sealed -> new родител2  
Drawing a generic shape... new -> РОДИТЕЛ  
Drawing a generic shape... base.  
Drawing a triangle...  
  
D:\Exercises\11 class\Inheritance\Shape_ove  
code 0.  
Press any key to close this window . . .
```